

PRODUCTION INSTALLATION RUNBOOK

Grafana Monitoring Platform

Two-Server Deployment on Red Hat Enterprise Linux 9

Grafana OSS • PostgreSQL 16 • Prometheus LTS • Alertmanager • Node Exporter

Prepared for on-premises DevOps application monitoring

Version 1.0 | 23 July 2026

Document purpose and deployment decision

This runbook provides a complete build, configuration, validation, backup, failover, and recovery procedure for a monitoring platform deployed on two RHEL 9 servers. It covers Grafana OSS, PostgreSQL, Prometheus, Alertmanager, Node Exporter, HAProxy, and NGINX.

Important architectural constraint: Two servers cannot provide safe automatic database quorum. This design therefore uses PostgreSQL primary/streaming-standby replication with controlled manual promotion. Prometheus and Alertmanager operate redundantly on both servers. Grafana is active on Server 1 and warm standby on Server 2. This is a production-ready active/standby design with documented recovery, not automatic active-active Grafana HA.

Item	Selected design
Operating system	Red Hat Enterprise Linux 9, x86_64
Grafana	Grafana OSS; active on Server 1, warm standby on Server 2
Grafana database	PostgreSQL 16 primary on Server 1, streaming standby on Server 2
Metrics	Prometheus 3.5.5 LTS on both servers
Notifications	Alertmanager 0.33.1 cluster on both servers
Host metrics	Node Exporter 1.12.1 on monitoring and application servers
Query failover	Local HAProxy selects the healthy Prometheus replica
User access	HTTPS through NGINX and a shared Grafana DNS name
Database failover	Manual promotion after fencing or isolating the failed primary

Scope

- Installation and service configuration on both monitoring servers.
- PostgreSQL database initialization, replication, backup, promotion, and re-seeding.
- Prometheus collection, retention controls, alert rules, and replica validation.
- Alertmanager clustering and notification integration placeholders.
- Grafana, Prometheus data-source provisioning, NGINX HTTPS, and standby recovery.
- Integration patterns for GitLab, Jenkins, SonarQube, Nexus, and Linux servers.
- Firewall, SELinux, authentication, backup, patching, monitoring, and acceptance tests.

Out of scope

- Automatic PostgreSQL failover without a third quorum member.
- A long-term deduplicating metrics store such as Thanos or Grafana Mimir.
- Organization-specific SSO, SMTP, Teams, ServiceNow, or PagerDuty credentials.
- Application changes that require approval from GitLab, Jenkins, SonarQube, or Nexus owners.

Source-controlled artifacts

Store the following files in an approved private configuration repository. Never commit passwords, API tokens, private keys, or unencrypted database credentials.

- /etc/prometheus/prometheus.yml and /etc/prometheus/rules/*.yml
- /etc/alertmanager/alertmanager.yml with secrets supplied at deployment time
- /etc/grafana/grafana.ini and /etc/grafana/provisioning/
- /etc/haproxy/haproxy.cfg and /etc/nginx/conf.d/grafana.conf
- Systemd unit files and the tested recovery procedure

1. Architecture and naming

1.1 Component placement

Component	Server 1	Server 2	Operating mode
PostgreSQL 16	Primary	Streaming standby	Manual promotion
Grafana OSS	Active	Warm standby	Start standby after DB promotion
Prometheus LTS	Replica A	Replica B	Both scrape all targets
Alertmanager	Replica A	Replica B	Clustered; both receive alerts
Node Exporter	Active	Active	Scraped by both Prometheus replicas
HAProxy	Active	Active	Local Prometheus primary/backup endpoint
NGINX	Active	Installed/stopped	HTTPS endpoint follows active Grafana

1.2 Replace these values

Before execution: Replace every angle-bracket value in this runbook. Do not paste a command containing <PLACEHOLDER> into a production shell.

Placeholder	Example	Purpose
<MON1_FQDN>	mon01.example.com	Monitoring Server 1 hostname
<MON1_IP>	10.10.10.11	Monitoring Server 1 address
<MON2_FQDN>	mon02.example.com	Monitoring Server 2 hostname
<MON2_IP>	10.10.10.12	Monitoring Server 2 address
<GRAFANA_FQDN>	grafana.example.com	User-facing Grafana DNS
<APPROVED_CIDR>	10.20.0.0/16	Authorized user network
<PROM_RETENTION_SIZE>	80GB	At most 80% of dedicated Prometheus disk
<SMTP_OR_WEBHOOK>	Organization supplied	Alert delivery destination
<BACKUP_PATH>	/backup/monitoring	Approved protected backup destination

1.3 Data flow

Users --HTTPS/443--> Active NGINX --> Active Grafana

```

|
+--> Local HAProxy/9091
|   |--> Prometheus A/9090
|   +--> Prometheus B/9090

```

Prometheus A and B --> Node Exporter and application metrics endpoints
 Prometheus A and B --> Alertmanager A and B --> Approved notification channel

Grafana Active --> PostgreSQL Primary
 PostgreSQL Primary --> streaming replication --> PostgreSQL Standby

1.4 Availability behavior

Failure	Expected behavior	Required action
One Prometheus	Collection and dashboards continue through the	Repair failed replica.

Failure	Expected behavior	Required action
replica	surviving replica.	
One Alertmanager replica	Surviving member sends notifications.	Repair failed member.
Server 2	Primary Grafana and PostgreSQL continue; monitoring loses redundancy.	Restore Server 2 promptly.
Server 1	Server 2 continues metric collection and alerting. Grafana is unavailable until controlled DB promotion.	Fence Server 1, promote PostgreSQL standby, start Grafana/NGINX, update DNS.
Network partition	Alertmanager may intentionally send duplicate notifications. PostgreSQL standby must never be promoted unless the old primary is isolated.	Follow incident command and fencing procedure.

2. Prerequisites and security approvals

2.1 Server minimums

Final capacity must be calculated from target count, active time-series count, scrape interval, retention, dashboard concurrency, and alert volume. The following is a starting point, not an automatic sizing approval.

Resource	Per server starting point	Notes
CPU	4-8 vCPU	Prometheus query and ingestion load is the main driver.
Memory	8-16 GB	Increase for high series cardinality or concurrent dashboards.
OS disk	40 GB	Separate from Prometheus and PostgreSQL data where possible.
Prometheus data	100 GB+ local SSD	Do not use NFS. Size retention to 80% or less.
PostgreSQL data	20 GB+ local SSD	Low volume for Grafana but include WAL and backups.
Network	Low-latency connection	Both nodes scrape all application targets.

2.2 Required organizational inputs

- Two registered and patched RHEL 9 servers with sudo access.
- Forward and reverse DNS for both monitoring servers.
- Shared Grafana DNS name and a documented DNS failover procedure.
- TLS server certificate and private key for the shared Grafana DNS name.
- PostgreSQL server certificates for both database nodes and the issuing CA certificate.
- Approved notification service and credentials.
- Firewall approvals for the port matrix below.
- Approved backup location with access controls and retention policy.
- Application-owner approval to enable metrics endpoints or install plugins/exporters.

2.3 Port matrix

Source	Destination	Port	Purpose
Approved user networks	Active Grafana/NGINX	443/TCP	Grafana HTTPS
Both Grafana nodes	Both Prometheus nodes	9090/TCP	Queries through HAProxy
Both Prometheus nodes	Both Alertmanagers	9093/TCP	Alert delivery
Alertmanager nodes	Peer Alertmanager	9094/ TCP+UDP	Cluster gossip
Both Prometheus nodes	All Node Exporters	9100/TCP	Host metrics
Both Prometheus nodes	PostgreSQL exporters if added	9187/TCP	Optional DB metrics
Server 2 PostgreSQL	Server 1 PostgreSQL	5432/TCP	Streaming replication
Both Grafana nodes	Active PostgreSQL	5432/TCP	Grafana database
Both Prometheus nodes	Application endpoints	Application-specific	GitLab/Jenkins/SonarQube/Nexus metrics

Security rule: Do not expose Prometheus, Alertmanager, Node Exporter, or application metrics endpoints to public or general user networks. Restrict them to the two monitoring server IP addresses and approved administrators.

3. Base RHEL 9 preparation

3.1 Run on both servers

```
cat /etc/redhat-release
uname -m
hostname -f
df -h
free -h
timedatectl
chronyc tracking
```

Confirm that `uname -m` returns `x86\_64`. The binary URLs in this runbook are for AMD64/x86_64.

```
sudo dnf update -y
sudo dnf install -y curl wget tar gzip unzip vim policycoreutils-python-utils  nginx haproxy
sudo systemctl reboot
```

Change control: Schedule and approve the RHEL patching and reboot. Reconnect after the server returns, then confirm the expected kernel and services.

3.2 Create data and backup mount points

Prefer dedicated local filesystems. The example retains the RHEL PostgreSQL default data location and creates an explicit backup directory.

```
sudo install -d -o root -g root -m 0750 /var/backups/monitoring
sudo install -d -o root -g root -m 0755 /etc/monitoring
df -hT /var/lib /var/backups/monitoring
```

3.3 Confirm SELinux and firewall state

```
getenforce
sudo firewall-cmd --state
sudo firewall-cmd --get-active-zones
```

Keep SELinux enforcing. Do not disable SELinux to solve a configuration error. Add only the required SELinux policy or boolean after confirming the denied operation.

4. Install PostgreSQL 16

Database role: Server 1 begins as PostgreSQL primary. Server 2 is initialized from Server 1 and remains a streaming standby until a controlled failover.

4.1 Install packages on both servers

```
sudo dnf module reset postgresql -y
sudo dnf module install postgresql:16/server -y
postgres --version
```

4.2 Initialize Server 1 only

```
sudo PGSETUP_INITDB_OPTIONS="--data-checksums" postgresql-setup --initdb
sudo -u postgres pg_controldata /var/lib/pgsql/data | grep "Data page checksum"
```

Expected result: `Data page checksum version` is non-zero.

4.3 Install PostgreSQL TLS files on Server 1

Copy the organization-issued Server 1 PostgreSQL certificate, private key, and issuing CA to the paths below.

```
sudo install -o postgres -g postgres -m 0600 <MON1_DB_PRIVATE_KEY> /var/lib/pgsql/data/server.key
sudo install -o postgres -g postgres -m 0644 <MON1_DB_CERTIFICATE> /var/lib/pgsql/data/server.crt
sudo install -o postgres -g postgres -m 0644 <DATABASE_CA_CERTIFICATE> /var/lib/pgsql/data/root.crt
sudo restorecon -RFv /var/lib/pgsql/data
```

4.4 Configure Server 1

Edit `/var/lib/pgsql/data/postgresql.conf` and set:

```
listen_addresses = 'localhost,<MON1_IP>'
port = 5432
password_encryption = 'scram-sha-256'
ssl = on
ssl_cert_file = 'server.crt'
ssl_key_file = 'server.key'
ssl_ca_file = 'root.crt'

wal_level = replica
max_wal_senders = 10
max_replication_slots = 10
wal_keep_size = '2GB'
hot_standby = on
max_connections = 200
shared_buffers = '512MB'
log_connections = on
log_disconnections = on
log_line_prefix = '%m [%p] %u@%d %r '
```

Append the following to `/var/lib/pgsql/data/pg\_hba.conf` after replacing the addresses:

```
hostssl grafana grafana <MON1_IP>/32 scram-sha-256
hostssl grafana grafana <MON2_IP>/32 scram-sha-256
hostssl replication replicator <MON2_IP>/32 scram-sha-256
```

4.5 Start Server 1 and create roles

```
sudo systemctl enable --now postgresql
sudo systemctl status postgresql --no-pager
sudo -u postgres psql -c "SELECT version();"
```

Open a PostgreSQL administrator session:

```
sudo -u postgres psql
```

At the `psql` prompt, run:

```
CREATE ROLE grafana LOGIN;
\password grafana

CREATE DATABASE grafana
  OWNER grafana
  ENCODING 'UTF8'
  TEMPLATE template0;

CREATE ROLE replicator WITH REPLICATION LOGIN;
\password replicator

\q
```

Password handling: Use strong, unique passwords supplied by the approved secret-management process. Do not place passwords directly in shell commands, tickets, source control, or this document.

4.6 Allow PostgreSQL through the firewall

On Server 1, allow TCP 5432 only from the two monitoring addresses. Replace the example addresses:

```
sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source address="<MON1_IP>/32" port protocol="tcp" port="5432"
accept'
sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source address="<MON2_IP>/32" port protocol="tcp" port="5432"
accept'
sudo firewall-cmd --reload
```

5. Build the PostgreSQL streaming standby

5.1 Prepare Server 2

Do not run `postgresql-setup --initdb` on Server 2. Stop PostgreSQL and confirm the target directory is empty before taking the base backup.

```
sudo systemctl stop postgresql
sudo install -d -o postgres -g postgres -m 0700 /var/lib/pgsql/data
sudo find /var/lib/pgsql/data -mindepth 1 -maxdepth 1 -print
```

Destructive check: If the command lists files, stop. Confirm that Server 2 contains no required PostgreSQL data before clearing or replacing the directory.

5.2 Create the replication password file

```
sudo -u postgres vi /var/lib/pgsql/.pgpass
```

Add one line:

```
<MON1_FQDN>:5432:replication:replicator:<REPLICATION_PASSWORD>
sudo chown postgres:postgres /var/lib/pgsql/.pgpass
sudo chmod 0600 /var/lib/pgsql/.pgpass
```

5.3 Take the base backup

```
sudo -u postgres pg_basebackup --host=<MON1_FQDN> --port=5432 --username=replicator --pgdata=/var/lib/pgsql/data --format=plain --wal-method=stream --progress --write-recovery-conf --create-slot --slot=grafana_standby
```

5.4 Replace Server 2 TLS identity

The base backup copies Server 1 configuration files. Replace the copied certificate and key with Server 2's identity before starting PostgreSQL.

```
sudo install -o postgres -g postgres -m 0600 <MON2_DB_PRIVATE_KEY> /var/lib/pgsql/data/server.key
sudo install -o postgres -g postgres -m 0644 <MON2_DB_CERTIFICATE> /var/lib/pgsql/data/server.crt
sudo install -o postgres -g postgres -m 0644 <DATABASE_CA_CERTIFICATE> /var/lib/pgsql/data/root.crt
sudo restorecon -RFv /var/lib/pgsql/data
```

Edit Server 2 `/var/lib/pgsql/data/postgresql.conf` and change:

```
listen_addresses = 'localhost,<MON2_IP>'
```

5.5 Start and validate the standby

```
sudo systemctl enable --now postgresql
sudo systemctl status postgresql --no-pager
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
```

Expected on Server 2: `pg\_is\_in\_recovery` returns `t`.

On Server 1:

```
sudo -u postgres psql -x -c "SELECT application_name, client_addr, state, sync_state, replay_lag FROM pg_stat_replication;"
```

Expected: one streaming replica. This design uses asynchronous replication to preserve Grafana availability if the standby is temporarily unavailable. Monitor replication lag and define the accepted recovery-point objective.

5.6 Verify read-only behavior

```
sudo -u postgres psql -d grafana -c "SELECT pg_is_in_recovery(), current_timestamp;"
```

6. Install Node Exporter

Install Node Exporter on both monitoring servers and later repeat the same procedure on every approved Linux application server.

6.1 Download and verify on each server

```
cd /tmp
curl -fLO https://github.com/prometheus/node_exporter/releases/download/v1.12.1/node_exporter-1.12.1.linux-amd64.tar.gz

echo "b51d8a76aa2a9156a55d501aca6276fae09e262259a5e4e831d2c2222f084e63 node_exporter-1.12.1.linux-amd64.tar.gz" | sha256sum -c -

tar -xzf node_exporter-1.12.1.linux-amd64.tar.gz
sudo useradd --system --no-create-home --shell /sbin/nologin node_exporter
sudo install -m 0755 node_exporter-1.12.1.linux-amd64/node_exporter /usr/local/bin/node_exporter
```

6.2 Create the service

```
sudo tee /etc/systemd/system/node_exporter.service >/dev/null <<'EOF'
[Unit]
Description=Prometheus Node Exporter
Wants=network-online.target
After=network-online.target

[Service]
User=node_exporter
Group=node_exporter
Type=simple
ExecStart=/usr/local/bin/node_exporter --web.listen-address=0.0.0.0:9100
Restart=on-failure
RestartSec=5
NoNewPrivileges=true
PrivateTmp=true
ProtectHome=true
ProtectSystem=strict

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now node_exporter
sudo systemctl status node_exporter --no-pager
curl -fsS http://localhost:9100/metrics | head
```

6.3 Firewall

On each Node Exporter host, allow TCP 9100 only from both Prometheus addresses.

```
sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source address="<MON1_IP>/32" port protocol="tcp" port="9100" accept'
sudo firewall-cmd --permanent --add-rich-rule='rule family="ipv4" source address="<MON2_IP>/32" port protocol="tcp" port="9100" accept'
sudo firewall-cmd --reload
```

7. Install Prometheus LTS on both servers

7.1 Download and verify

```
cd /tmp
curl -fLO https://github.com/prometheus/prometheus/releases/download/v3.5.5/prometheus-3.5.5.linux-amd64.tar.gz

echo "64d0beab873272b861a91df41668bc852c7e2e5b23f75c16059fb15b5630c577 prometheus-3.5.5.linux-amd64.tar.gz" | sha256sum -c -

tar -xzf prometheus-3.5.5.linux-amd64.tar.gz
sudo useradd --system --no-create-home --shell /sbin/nologin prometheus
sudo install -d -o prometheus -g prometheus -m 0750 /etc/prometheus /etc/prometheus/rules /var/lib/prometheus
sudo install -m 0755 prometheus-3.5.5.linux-amd64/prometheus /usr/local/bin/prometheus
sudo install -m 0755 prometheus-3.5.5.linux-amd64/promtool /usr/local/bin/promtool
```

7.2 Prometheus configuration

Create `/etc/prometheus/prometheus.yml` on both servers. On Server 2 change the replica label from `prometheus-a` to `prometheus-b`.

```
global:
  scrape_interval: 30s
  scrape_timeout: 10s
  evaluation_interval: 30s
  external_labels:
    environment: production
    replica: prometheus-a

rule_files:
  - /etc/prometheus/rules/*.yml

alerting:
  alertmanagers:
    - static_configs:
        - targets:
            - <MON1_IP>:9093
            - <MON2_IP>:9093

scrape_configs:
  - job_name: prometheus
    static_configs:
      - targets:
          - <MON1_IP>:9090
          - <MON2_IP>:9090

  - job_name: monitoring-servers
    static_configs:
      - targets:
          - <MON1_IP>:9100
          - <MON2_IP>:9100

  - job_name: application-linux-servers
    static_configs:
      - targets:
          - <GITLAB_SERVER>:9100
          - <JENKINS_SERVER>:9100
          - <SONARQUBE_SERVER>:9100
          - <NEXUS_SERVER>:9100
```

7.3 Initial alert rules

Create `/etc/prometheus/rules/infrastructure.yml` on both servers:

```

groups:
- name: infrastructure
  rules:
  - alert: TargetDown
    expr: up == 0
    for: 5m
    labels:
      severity: critical
    annotations:
      summary: "Monitoring target is unavailable"
      description: "{{ $labels.job }} target {{ $labels.instance }} is down."

  - alert: HostHighCPU
    expr: 100 - (avg by(instance)
      (rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) > 90
    for: 15m
    labels:
      severity: warning
    annotations:
      summary: "High CPU usage on {{ $labels.instance }}"

  - alert: HostLowMemory
    expr: (node_memory_MemAvailable_bytes /
      node_memory_MemTotal_bytes) * 100 < 10
    for: 10m
    labels:
      severity: warning
    annotations:
      summary: "Available memory below 10% on {{ $labels.instance }}"

  - alert: HostFilesystemLow
    expr: 100 * node_filesystem_avail_bytes{fstype!~"tmpfs|overlay"} /
      node_filesystem_size_bytes{fstype!~"tmpfs|overlay"} < 15
    for: 15m
    labels:
      severity: warning
    annotations:
      summary: "Filesystem space below 15% on {{ $labels.instance }}"

```

7.4 Validate ownership and syntax

```

sudo chown -R prometheus:prometheus /etc/prometheus /var/lib/prometheus
sudo promtool check config /etc/prometheus/prometheus.yml
sudo promtool check rules /etc/prometheus/rules/infrastructure.yml

```

7.5 Create the Prometheus service

Retention value: Replace <PROM_RETENTION_SIZE> with no more than approximately 80% of the dedicated Prometheus filesystem. Example: use 80GB only when at least 100GB is allocated exclusively to Prometheus.

```
sudo tee /etc/systemd/system/prometheus.service >/dev/null <<'EOF'
[Unit]
Description=Prometheus Monitoring Server
Wants=network-online.target
After=network-online.target

[Service]
User=prometheus
Group=prometheus
Type=simple
ExecStart=/usr/local/bin/prometheus --config.file=/etc/prometheus/prometheus.yml --storage.tsdb.path=/var/lib/prometheus --
storage.tsdb.retention.time=30d --storage.tsdb.retention.size=<PROM_RETENTION_SIZE> --web.listen-address=0.0.0.0:9090
Restart=on-failure
RestartSec=5
LimitNOFILE=65536
NoNewPrivileges=true
PrivateTmp=true
ProtectHome=true
ProtectSystem=full
ReadWritePaths=/var/lib/prometheus

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now prometheus
sudo systemctl status prometheus --no-pager
curl -fsS http://localhost:9090/-/healthy
curl -fsS http://localhost:9090/-/ready
```

7.6 Network access

Allow TCP 9090 between the two monitoring servers. Do not expose the Prometheus UI broadly.

8. Install Alertmanager on both servers

8.1 Download and verify

```
cd /tmp
curl -fLO https://github.com/prometheus/alertmanager/releases/download/v0.33.1/alertmanager-0.33.1.linux-amd64.tar.gz

echo "93d802cba6a8d27 239d747ce117df7648d326ab67 394e32 247 540b030e9842ba alertmanager-0.33.1.linux-amd64.tar.gz" | sha256sum -c -

tar -xzf alertmanager-0.33.1.linux-amd64.tar.gz
sudo useradd --system --no-create-home --shell /sbin/nologin alertmanager
sudo install -d -o alertmanager -g alertmanager -m 0750 /etc/alertmanager /var/lib/alertmanager
sudo install -m 0755 alertmanager-0.33.1.linux-amd64/alertmanager /usr/local/bin/alertmanager
sudo install -m 0755 alertmanager-0.33.1.linux-amd64/amtool /usr/local/bin/amtool
```

8.2 Configure routing

Create `/etc/alertmanager/alertmanager.yml` on both servers. The initial null receiver is valid for build validation but must be replaced before go-live.

```
global:
  resolve_timeout: 5m

route:
  receiver: operations-placeholder
  group_by: [alertname, job]
  group_wait: 30s
  group_interval: 5m
  repeat_interval: 4h

receivers:
  - name: operations-placeholder
```

Go-live gate: Production acceptance must fail until a test alert is successfully delivered to the approved notification channel and acknowledged by the support team.

8.3 Validate configuration

```
sudo chown -R alertmanager:alertmanager /etc/alertmanager /var/lib/alertmanager
sudo amtool check-config /etc/alertmanager/alertmanager.yml
```

8.4 Server 1 service

```
sudo tee /etc/systemd/system/alertmanager.service >/dev/null <<'EOF'
[Unit]
Description=Prometheus Alertmanager
Wants=network-online.target
After=network-online.target

[Service]
User=alertmanager
Group=alertmanager
Type=simple
ExecStart=/usr/local/bin/alertmanager --config.file=/etc/alertmanager/alertmanager.yml --storage.path=/var/lib/alertmanager --
web.listen-address=0.0.0.0:9093 --cluster.listen-address=0.0.0.0:9094 --cluster.advertise-address=<MON1_IP>:9094 --
cluster.peer=<MON2_IP>:9094
Restart=on-failure
RestartSec=5
NoNewPrivileges=true
PrivateTmp=true
ProtectHome=true
ProtectSystem=full
ReadWritePaths=/var/lib/alertmanager

[Install]
WantedBy=multi-user.target
EOF
```

8.5 Server 2 service

Use the same service, changing only:

```
--cluster.advertise-address=<MON2_IP>:9094
--cluster.peer=<MON1_IP>:9094
```

8.6 Start and validate

```
sudo systemctl daemon-reload
sudo systemctl enable --now alertmanager
sudo systemctl status alertmanager --no-pager
curl -fsS http://localhost:9093/-/healthy
curl -fsS http://localhost:9093/-/ready
curl -fsS http://localhost:9093/api/v2/status
```

Allow 9093/TCP from both Prometheus nodes. Allow 9094/TCP and UDP only between the two Alertmanager nodes.

9. Configure Prometheus query failover

Grafana queries a local HAProxy listener on `127.0.0.1:9091`. HAProxy uses the local Prometheus replica first and the remote replica as backup.

9.1 Server 1 HAProxy configuration

```
global
  log /dev/log local0
  user haproxy
  group haproxy
  daemon

defaults
  log global
  mode http
  timeout connect 5s
  timeout client 60s
  timeout server 60s

frontend prometheus_query
  bind 127.0.0.1:9091
  default_backend prometheus_servers

backend prometheus_servers
  option httpchk GET /-/ready
  http-check expect status 200
  server prometheus-a <MON1_IP>:9090 check
  server prometheus-b <MON2_IP>:9090 check backup
```

9.2 Server 2 HAProxy configuration

Use the same file but reverse primary and backup:

```
server prometheus-b <MON2_IP>:9090 check
server prometheus-a <MON1_IP>:9090 check backup
```

9.3 Validate and start

```
sudo haproxy -c /etc/haproxy/haproxy.cfg
sudo systemctl enable --now haproxy
sudo systemctl status haproxy --no-pager
curl -fsS http://127.0.0.1:9091/-/ready
```

10. Install Grafana OSS on both servers

10.1 Add the official repository

```
cd /tmp
wget -q -O grafana-gpg.key https://rpm.grafana.com/gpg.key
sudo rpm --import grafana-gpg.key

sudo tee /etc/yum.repos.d/grafana.repo >/dev/null <<'EOF'
[grafana]
name=grafana
baseurl=https://rpm.grafana.com
repo_gpgcheck=1
enabled=1
gpgcheck=1
gpgkey=https://rpm.grafana.com/gpg.key
sslverify=1
sslcert=/etc/pki/tls/certs/ca-bundle.crt
EOF

sudo dnf makecache --refresh
sudo dnf install -y grafana
rpm -q grafana
```

10.2 Create shared Grafana secrets

The database password and Grafana `secret\_key` must be identical on both servers. Store them through the organizational secret-management process.

```
sudo install -d -o root -g grafana -m 0750 /etc/grafana/secrets
sudo vi /etc/grafana/secrets/postgres_password
sudo vi /etc/grafana/secrets/secret_key
sudo chown root:grafana /etc/grafana/secrets/*
sudo chmod 0640 /etc/grafana/secrets/*
```

Generate the secret key on an approved secure workstation, store it in the secret manager, and deploy the same value to both nodes. Do not regenerate it during failover.

10.3 Install the database CA

```
sudo install -o root -g grafana -m 0640 <DATABASE_CA_CERTIFICATE> /etc/grafana/postgresql-ca.crt
```

10.4 Configure Server 1 Grafana

Edit `/etc/grafana/grafana.ini`:

```
[database]
type = postgres
host = <MON1_FQDN>:5432
name = grafana
user = grafana
password = $__file{/etc/grafana/secrets/postgres_password}
ssl_mode = verify-full
ca_cert_path = /etc/grafana/postgresql-ca.crt
max_idle_conn = 10
max_open_conn = 100
conn_max_lifetime = 3600
migration_locking = true
locking_attempt_timeout_sec = 60

[server]
protocol = http
http_addr = 127.0.0.1
http_port = 3000
domain = <GRAFANA_FQDN>
root_url = https://<GRAFANA_FQDN>/

[security]
secret_key = $__file{/etc/grafana/secrets/secret_key}
cookie_secure = true
cookie_samesite = strict

[users]
allow_sign_up = false

[auth.anonymous]
enabled = false

[auth.basic]
enabled = true
password_policy = true

[unified_alerting]
enabled = false
```

10.5 Configure Server 2 Grafana

Use the same file. While Server 1 is primary, Server 2 also points to ``<MON1_FQDN>:5432``. Keep Server 2 Grafana stopped until failover testing or recovery.

10.6 Provision Prometheus

```
sudo tee /etc/grafana/provisioning/datasources/prometheus.yml >/dev/null <<'EOF'
apiVersion: 1

datasources:
- name: Prometheus
  type: prometheus
  access: proxy
  url: http://127.0.0.1:9091
  isDefault: true
  editable: false
EOF
```

10.7 Start Grafana on Server 1

```
sudo systemctl daemon-reload
sudo systemctl enable --now grafana-server
sudo systemctl status grafana-server --no-pager
curl -fsS http://127.0.0.1:3000/api/health
```

Expected: database status is `ok`. Review logs if startup fails:

```
sudo journalctl -u grafana-server --since "30 minutes ago" --no-pager
```

10.8 Keep Server 2 in warm standby

```
sudo systemctl disable --now grafana-server
```

11. Configure NGINX and HTTPS

11.1 Install the shared Grafana certificate on both servers

```
sudo install -o root -g root -m 0644 <GRAFANA_TLS_CERTIFICATE> /etc/pki/tls/certs/grafana.crt
sudo install -o root -g root -m 0600 <GRAFANA_TLS_PRIVATE_KEY> /etc/pki/tls/private/grafana.key
```

11.2 Create NGINX configuration on both servers

```
sudo tee /etc/nginx/conf.d/grafana.conf >/dev/null <<'EOF'
map $http_upgrade $connection_upgrade {
    default upgrade;
    "" close;
}

server {
    listen 443 ssl;
    server_name <GRAFANA_FQDN>;

    ssl_certificate /etc/pki/tls/certs/grafana.crt;
    ssl_certificate_key /etc/pki/tls/private/grafana.key;
    ssl_protocols TLSv1.2 TLSv1.3;

    location / {
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto https;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;
        proxy_pass http://127.0.0.1:3000;
    }
}
EOF

sudo setsebool -P httpd_can_network_connect 1
sudo nginx -t
```

11.3 Start NGINX on Server 1

```
sudo systemctl enable --now nginx
sudo systemctl status nginx --no-pager
sudo firewall-cmd --permanent --add-service=https
sudo firewall-cmd --reload
```

Keep NGINX installed but stopped on Server 2 until controlled failover:

```
sudo systemctl disable --now nginx
```

11.4 DNS

Point `<GRAFANA_FQDN>` to Server 1 for normal operations. Use a low but policy-compliant TTL to support controlled recovery. Document the owner and exact DNS change procedure.

11.5 Initial login

Open `https://<GRAFANA_FQDN>/`. Sign in with `admin/admin`, then immediately replace the default password. Configure approved users and roles. If SSO is required, complete and test the organization's supported identity-provider integration before go-live.

12. Integrate application metrics

12.1 Linux host metrics

Install Node Exporter on each GitLab, Jenkins, SonarQube, and Nexus Linux server. Restrict port 9100 to the two Prometheus IP addresses. Add each target to both Prometheus configurations.

12.2 GitLab Self-Managed

- Confirm the GitLab installation method and version.
- Use GitLab's built-in Prometheus metrics and bundled exporters where supported.
- Allow only the two Prometheus server addresses in the monitoring allowlist.
- Do not expose unauthenticated exporters outside the monitoring network.
- Collect GitLab, Sidekiq, Gitaly, Workhorse, PostgreSQL, Redis, Registry, and node metrics as applicable.

12.3 Jenkins

- Install the Jenkins Prometheus Metrics plugin through the approved plugin process.
- Confirm the plugin supports the installed Jenkins controller version.
- Validate the default `/prometheus/` endpoint, including its required trailing slash.
- Use Jenkins authentication and network restrictions consistent with organizational policy.
- Validate job status, duration, executor, queue, JVM, and controller metrics before dashboard acceptance.

12.4 SonarQube

- Use `/api/monitoring/metrics` for supported operational metrics and secure it with the configured Sonar system passcode.
- Use `/api/measures` for project code-quality values and histories.
- Because code-quality API responses are not identical to ordinary Prometheus scraping, use an approved API exporter or collection process where required.

- Also collect CPU, memory, disk, and Java process metrics through Node Exporter and approved JMX collection.

12.5 Nexus Repository

- For Nexus Repository 3.81 or later, use `/service/rest/metrics/prometheus``.
- For earlier supported releases, validate `/service/metrics/prometheus``.
- Use a dedicated account with the ``nx-metrics-all`` privilege.
- Use synthetic HTTP checks for required artifact or repository availability because native instance metrics may not prove that a specific artifact can be downloaded.

12.6 Safe rollout procedure

1. Enable one application endpoint at a time.
2. Validate the endpoint locally on the application server.
3. Validate network access from both Prometheus servers.
4. Add the target to both Prometheus configurations.
5. Run ``promtool check config`` on both nodes.
6. Restart both Prometheus services during the approved window.
7. Confirm the target is ``UP`` on both replicas.
8. Validate metrics with the application owner before creating alerts.

13. Backups and recovery

13.1 What must be backed up

Artifact	Backup method	Frequency
Grafana PostgreSQL database	Nightly <code>`pg_dump`</code> plus retention in approved backup storage	Daily and before upgrades
PostgreSQL configuration	Protected file backup	After every approved change
Grafana configuration and provisioning	Protected backup and source control without secrets	After every change
Grafana plugins	Package/plugin inventory; reinstall from approved source	After plugin change
Prometheus/Alertmanager configuration	Private source control	After every change
Prometheus data	Redundant replicas; optional snapshots if history requires backup	Policy dependent
TLS certificates and keys	PKI/secret-management system	According to PKI policy

13.2 PostgreSQL logical backup

Run on Server 1 using an approved protected path:

```
sudo -u postgres pg_dump --format=custom --file=<BACKUP_PATH>/grafana_$(date +%F_%H%M).dump grafana
sudo -u postgres pg_restore --list <BACKUP_PATH>/<GRAFANA_BACKUP_FILE>.dump | head
```

Copy backups off Server 1. Streaming replication is not a substitute for backup because accidental deletions and corruption can replicate.

13.3 Configuration backup

```
sudo tar --xattrs --selinux -czf <BACKUP_PATH>/monitoring-config_$(date +%F_%H%M).tgz /etc/grafana /etc/prometheus
/etc/alertmanager /etc/haproxy /etc/nginx/conf.d/grafana.conf /etc/systemd/system/prometheus.service
/etc/systemd/system/alertmanager.service /etc/systemd/system/node_exporter.service
```

Sensitive content: The archive contains secrets and private configuration. Encrypt it, tightly restrict access, and never attach it to an ordinary ticket.

13.4 Restore test

- Perform a scheduled restore to a non-production PostgreSQL instance.
- Validate Grafana startup against the restored database.
- Confirm dashboards, users, data sources, and permissions.
- Record restoration time and compare it with the recovery-time objective.
- Correct the procedure after every failed or incomplete restore test.

14. Controlled failover to Server 2

Critical safety requirement: Never promote Server 2 while Server 1 PostgreSQL may still accept writes. First fence, power off, or network-isolate Server 1. This prevents split-brain and divergent Grafana databases.

14.1 Decision and fencing

1. Declare the monitoring-platform incident and assign an incident commander.
2. Confirm Server 1 is unavailable and not merely isolated from the operator.
3. Fence or power off Server 1, or block its PostgreSQL and Grafana network access.
4. Record the last known PostgreSQL replication lag and accepted data-loss risk.
5. Approve promotion of Server 2.

14.2 Promote PostgreSQL on Server 2

```
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
sudo -u postgres psql -c "SELECT pg_promote(wait_seconds => 60);"
sudo -u postgres psql -c "SELECT pg_is_in_recovery();"
```

Expected after promotion: `pg\_is\_in\_recovery` returns `f`.

14.3 Point Server 2 Grafana to the new primary

Edit Server 2 `/etc/grafana/grafana.ini`:

```
[database]
host = <MON2_FQDN>:5432

sudo systemctl enable --now grafana-server
sudo systemctl enable --now nginx
sudo systemctl status grafana-server nginx --no-pager
curl -fsS http://127.0.0.1:3000/api/health
```

14.4 Update DNS

Change ``<GRAFANA_FQDN>`` to Server 2 only after local validation succeeds. Confirm HTTPS, login, dashboards, data sources, and permissions.

14.5 Rebuild Server 1 as standby

After Server 1 is repaired, do not simply restart its old PostgreSQL database. Re-seed it from the new primary using a fresh `pg\_basebackup`, or use `pg\_rewind` only under a separately tested DBA procedure. A fresh base backup is slower but operationally simpler and safer.

14.6 Return to normal roles

Fail back only during an approved maintenance window. Rebuild Server 1 as standby, validate replication, decide whether Server 2 remains primary, and avoid unnecessary role reversal.

15. Production validation and acceptance

15.1 Service validation

```
sudo systemctl is-active postgresql
sudo systemctl is-active prometheus
sudo systemctl is-active alertmanager
sudo systemctl is-active node_exporter
sudo systemctl is-active haproxy

curl -fsS http://localhost:9090/-/ready
curl -fsS http://localhost:9093/-/ready
curl -fsS http://localhost:9100/metrics | head
curl -fsS http://127.0.0.1:9091/-/ready
```

15.2 Acceptance checklist

- ☐ Both Prometheus replicas show all required targets as UP.
- ☐ Both Prometheus replicas evaluate the same alert-rule set.
- ☐ Both Alertmanager nodes show the expected two-member cluster.
- ☐ A test alert is delivered and acknowledged through the approved notification channel.
- ☐ Grafana reports database status `ok` and loads dashboards through HAProxy.
- ☐ Stopping Prometheus A causes Grafana queries to continue through Prometheus B.
- ☐ Stopping Alertmanager A does not prevent notification delivery.
- ☐ PostgreSQL standby reports `pg\_is\_in\_recovery() = true` and acceptable replay lag.
- ☐ A controlled PostgreSQL promotion test has been completed in a safe test window.
- ☐ Grafana DNS and TLS certificate validation succeed.
- ☐ Default administrator credentials have been replaced.
- ☐ Anonymous access and user self-registration are disabled.
- ☐ All exporter and metrics ports are restricted to approved monitoring sources.
- ☐ Backups complete successfully and a restoration test has passed.
- ☐ The operations team has the runbook, ownership matrix, and escalation contacts.

15.3 Failure tests

Test	Expected result	Evidence
Stop local Prometheus	HAProxy uses remote replica; dashboards continue.	HAProxy status/log and Grafana query
Stop one Alertmanager	Surviving member sends test notification.	Notification and cluster status
Block one Node Exporter	TargetDown fires after configured delay.	Prometheus alert and notification

Test	Expected result	Evidence
Restart monitoring node	Enabled services return automatically.	Systemd status and logs
PostgreSQL standby lag	Alert before RPO is exceeded.	Replication dashboard and alert
Restore PostgreSQL backup	Grafana starts with expected data.	Restore record and screenshots

16. Operations, patching, and governance

16.1 Daily checks

- Target availability and scrape errors.
- Prometheus disk usage, ingestion rate, query latency, and rule failures.
- Alertmanager cluster membership and notification errors.
- PostgreSQL replication state and replay lag.
- Grafana health, failed logins, data-source errors, and certificate expiry.

16.2 Upgrade sequence

1. Review release notes, security advisories, compatibility, and rollback requirements.
2. Back up PostgreSQL and all configuration files.
3. Upgrade and validate Server 2 monitoring components first.
4. Confirm Prometheus and Alertmanager redundancy remains healthy.
5. Upgrade Server 1 during the approved window.
6. For Grafana, upgrade the standby package first but keep it stopped.
7. Upgrade active Grafana only after a verified database backup.
8. Do not downgrade Grafana without restoring the pre-upgrade database backup.

16.3 Configuration change sequence

```
sudo promtool check config /etc/prometheus/prometheus.yml
sudo promtool check rules /etc/prometheus/rules/*.yml
sudo amtool check-config /etc/alertmanager/alertmanager.yml
sudo haproxy -c -f /etc/haproxy/haproxy.cfg
sudo nginx -t
```

Deploy to Server 2 first, validate, then deploy the identical functional change to Server 1. Preserve only the expected node-specific values, such as replica labels and peer addresses.

16.4 Ownership

Area	Primary owner	Required responsibilities
RHEL and storage	Linux/platform team	Patching, capacity, filesystem, service recovery
PostgreSQL	Database/platform team	Replication, backup, promotion, recovery
Grafana	Monitoring team	Users, dashboards, provisioning, upgrades
Prometheus/ Alertmanager	Monitoring team	Scrapes, rules, retention, notifications
Application endpoints	Application owners	Metrics enablement and version compatibility
Network/PKI/DNS	Infrastructure teams	Firewall, certificates, names, failover
Incident response	Operations	Acknowledgment, escalation, runbook execution

Appendix A — Command and configuration inventory

Component	Configuration	Data	Service
PostgreSQL	/var/lib/pgsql/data/*.conf	/var/lib/pgsql/data	postgresql
Grafana	/etc/grafana/grafana.ini	PostgreSQL database	grafana-server
Grafana provisioning	/etc/grafana/provisioning	N/A	grafana-server
Prometheus	/etc/prometheus	/var/lib/prometheus	prometheus
Alertmanager	/etc/alertmanager	/var/lib/alertmanager	alertmanager
Node Exporter	systemd unit	N/A	node_exporter
HAProxy	/etc/haproxy/haproxy.cfg	N/A	haproxy
NGINX	/etc/nginx/conf.d/grafana.conf	TLS files	nginx

Appendix B — Troubleshooting quick reference

Symptom	Checks
Grafana database error	Test PostgreSQL TLS/login; inspect Grafana logs; verify CA, DNS, password file, pg_hba.conf.
Grafana dashboard has no data	Test HAProxy 9091; check both Prometheus readiness endpoints; inspect target status.
Prometheus will not start	Run promtool; check ownership, retention value, disk space, and systemd logs.
Target DOWN	Test endpoint from both Prometheus nodes; check firewall, authentication, certificate, and application health.
Duplicate notifications	Inspect Alertmanager cluster membership and network partition; duplicates may be intentional fail-open behavior.
PostgreSQL standby not streaming	Check `pgpass`, replication slot, pg_hba.conf, certificate, port 5432, and primary logs.
NGINX 502	Confirm Grafana is listening on 127.0.0.1:3000 and SELinux boolean permits proxy connection.
Standby Grafana cannot decrypt secrets	Confirm the same Grafana secret_key file is deployed to both nodes.

Appendix C — Authoritative references

Grafana RHEL installation: <https://grafana.com/docs/grafana/latest/setup-grafana/installation/redhat-rhel-fedora/>

Grafana configuration: <https://grafana.com/docs/grafana/latest/setup-grafana/configure-grafana/>

Grafana high availability: <https://grafana.com/docs/grafana/latest/setup-grafana/set-up-for-high-availability/>

Grafana backup: <https://grafana.com/docs/grafana/latest/administration/back-up-grafana/>

Prometheus downloads: <https://prometheus.io/download/>

Prometheus storage: <https://prometheus.io/docs/prometheus/latest/storage/>

Prometheus security model: <https://prometheus.io/docs/operating/security/>

Alertmanager high availability: https://prometheus.io/docs/alerting/latest/high_availability/

RHEL 9 PostgreSQL installation: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/9/html/configuring_and_using_database_servers/using-postgresql_configuring-and-using-database-servers

PostgreSQL streaming replication: <https://www.postgresql.org/docs/16/warm-standby.html>

PostgreSQL base backup: <https://www.postgresql.org/docs/current/app-pgbasebackup.html>

GitLab Prometheus monitoring: <https://docs.gitlab.com/administration/monitoring/prometheus/>

Jenkins Prometheus plugin: <https://plugins.jenkins.io/prometheus/>

SonarQube Web API: <https://docs.sonarsource.com/sonarqube-server/extension-guide/web-api>

Nexus Prometheus metrics: <https://help.sonatype.com/en/prometheus.html>

Version note: This runbook selected Prometheus 3.5.5 LTS, Alertmanager 0.33.1, and Node Exporter 1.12.1 based on the official Prometheus download catalog available on 23 July 2026. Revalidate versions and checksums immediately before production installation.